Complete additional program for the Livingstone incidence construction

This is the full new mathematical program required for the Livingstone application. The existing Topic C construction and exhaustive census programs remain charged in full in their original appendix. The shared construction file is not repeated here. No branch of the new program is removed. The complete Topic C software cost is the old full appendix together with this appendix, or the three entire raw program files as an alternative representation. Operational drivers and partial Lean evidence are separate.

```
# Exact L068 source-object reconstruction. No source geometry census is input.
# The literal first definition is refuted; its explicit repair and P are checked.
LoadPackage("tomlib");;
SizeScreen([100000,24]);;
Check179:=function(b,s) if not b then Error(s); fi; end;;
G:=UnderlyingGroup(TableOfMarks("J1"));;
Check179(Size(G)=175560 and LargestMovedPoint(G)=266,"J1 action");;
V:=[1..266];;
Hv:=Stabilizer(G,1);;
ovs:=Orbits(Hv,V);;
Check179(SortedList(List(ovs,Length))=[1,11,12,110,132],"point suborbits");;
nb:=Set(First(ovs,x->Length(x)=11));;
Adj:=List(V,v->OnSets(nb,RepresentativeAction(G,1,v)));;
Check179(ForAll(V,v->Length(Adj[v])=11 and not v in Adj[v]
 and ForAll(Adj[v],u->v in Adj[u])),"undirected 11-regular graph");;
Check179(ForAll(GeneratorsOfGroup(G),g->ForAll(V,v->
 OnSets(Adj[v],g)=Adj[v^g])),"graph invariance");;
Dist:=List(V,v->List(V,x->-1));;
for v in V do
 Dist[v][v]:=0; frontier:=[v]; step:=0;
 while Length(frontier)>0 do
  step:=step+1; nxt:=Difference(Union(Adj{frontier}),
   Filtered(V,x->Dist[v][x]<>-1));
  for x in nxt do Dist[v][x]:=step; od;
  frontier:=nxt;
 od;
od;
Check179(ForAll(Dist,row->List([0..4],d->Number(row,x->x=d))
 =[1,11,110,132,12]),"distance distribution");;
k:=0;;
for v in V do for u in V do
 k:=Dist[v][u];
 if k>0 then
  Check179(Number(Adj[u],x->Dist[v][x]=k-1)=[1,1,5,11][k],"intersection c_i");
 fi;
 if k<4 then
  Check179(Number(Adj[u],x->Dist[v][x]=k+1)=[11,10,6,1][k+1],"intersection b_i");
 fi;
od;od;
Edges:=Set(Concatenation(List(V,v->List(Filtered(Adj[v],u->u>v),u->[v,u]))));;
Check179(Length(Edges)=1463,"edge count");;
Pentagons:=[];;
for a in V do for b in Filtered(Adj[a],x->x>a) do
 for c in Filtered(Adj[b],x->x>a) do
  for d in Filtered(Adj[c],x->x>a and x<>b) do
   for e in Filtered(Intersection(Adj[d],Adj[a]),x->x>b and x<>c) do
    Add(Pentagons,Set([a,b,c,d,e]));
   od;
  od;
 od;
od;od;
Pentagons:=Set(Pentagons);;
Check179(Length(Pentagons)=11704,"all pentagons");;
co:=OrbitsDomain(G,Pentagons,OnSets);;
Check179(SortedList(List(co,Length))=[2926,8778],"pentagon orbits");;
White:=Set(First(co,x->Length(x)=2926));;
Orange:=Set(First(co,x->Length(x)=8778));;
He:=Stabilizer(G,Edges[1],OnSets);;
eo:=Orbits(He,V);;
Check179(SortedList(List(eo,Length))=[2,10,20,24,30,60,120],"edge suborbits");;
BasePet:=Set(First(eo,x->Length(x)=10));;
Pet:=List(Edges,e->OnSets(BasePet,RepresentativeAction(G,Edges[1],e,OnSets)));;
```

```
Check179(Length(Set(Pet))=1463,"edge-Petersen bijection");;
for p in Pet do
 Check179(ForAll(p,v->Length(Intersection(Adj[v],p))=3),"Petersen cubic");
 for e in Combinations(p,2) do
  if e[2] in Adj[e[1]] then target:=0; else target:=1; fi;
  Check179(Length(Intersection(Adj[e[1]],Adj[e[2]],p))=target,"Petersen lambda/mu");
 od;
od;
PetWhite:=List(Pet,p->Filtered([1..Length(White)],i->IsSubset(p,White[i])));;
Check179(ForAll(PetWhite,x->Length(x)=6),"six white pentagons per Petersen");;
WhitePets:=List(White,w->[]);;
for k in [1..Length(Pet)] do
 for i in PetWhite[k] do
  Add(WhitePets[i],k);
  Check179(Difference(Pet[k],White[i]) in Orange,"orange complement");
 od;
od;
Check179(ForAll(WhitePets,x->Length(x)=3),"three Petersens per white pentagon");;
Quads:=[];;
for k in [1..Length(Pet)] do for i in PetWhite[k] do
 Add(Quads,[White[i],Difference(Pet[k],White[i]),Edges[k],
  Set(Edges{Difference(WhitePets[i],[k])})]);
od;od;
Quads:=Set(Quads);;
QuadAction:=function(q,g)
 return [OnSets(q[1],g),OnSets(q[2],g),OnSets(q[3],g),
  Set(List(q[4],e->OnSets(e,g)))];
end;;
Check179(Length(Quads)=8778 and IsTransitive(G,Quads,QuadAction),"quadruple orbit");;
Print("GRAPH vertices ",Length(V)," edges ",Length(Edges)," cycles ",Length(Pentagons),
 " white ",Length(White)," orange ",Length(Orange)," Petersens ",Length(Pet),
 " quadruples ",Length(Quads),"\n");
Print("PROPOSITION_3_3_LITERAL_COMMON_DISTANCE_TWO ",
 Number(V,v->Dist[Edges[1][1]][v]=2 and Dist[Edges[1][2]][v]=2),"\n");
Print("STAGE_GRAPH_PASS\n");
# Completeness of the Petersen family: extend a pentagon by its five spokes.
# Girth five forces each next outside vertex once one spoke is chosen.
PetersenExtensions179:=function(p)
 local cyc,nxt,extensions,x,ext,j,cand,q;
 cyc:=[p[1],First(Adj[p[1]],x->x in p)];
 while Length(cyc)<5 do
  nxt:=First(Intersection(Adj[cyc[Length(cyc)]],p),x->not x in cyc);
  Add(cyc,nxt);
 od;
 extensions:=[];
 for x in Difference(Adj[cyc[1]],p) do
  ext:=[x];
  for j in [3,5,2,4] do
   cand:=Difference(Intersection(Adj[Last(ext)],Adj[cyc[j]]),p);
   Check179(Length(cand)<=1,"girth-five spoke uniqueness");
   if Length(cand)<>1 then break; fi;
   if cand[1] in ext then break; fi;
   Add(ext,cand[1]);
  od;
  if Length(ext)=5 and ext[1] in Adj[ext[5]] then
   q:=Union(p,ext);
   Check179(Length(q)=10 and ForAll(q,v->Length(Intersection(Adj[v],q))=3),
    "spoke extension induced Petersen");
   AddSet(extensions,q);
  fi;
 od;
 return extensions;
end;;
for p in [White[1],Orange[1]] do
 ext:=PetersenExtensions179(p);
 if p in White then target:=3; else target:=1; fi;
 Check179(Length(ext)=target and ForAll(ext,q->q in Pet),"all Petersen extensions");
od;
corrected:=Set(Concatenation(List(Filtered(White,w->IsSubset(w,Edges[1])),
 w->Filtered(w,v->ForAll(Edges[1],u->Dist[u][v]=2)))));;
Check179(corrected=Pet[1],"white-pentagon corrected edge-Petersen rule");;
for v in Edges[1] do
 Check179(ForAll(Pet[1],z->Length(Intersection(Adj[v],Adj[z]))=1),"unique neighbour
 ↪  correspondence");
```

```
 Check179(Set(List(Pet[1],z->Intersection(Adj[v],Adj[z])[1]))
  =Difference(Adj[v],Edges[1]),"neighbour correspondence bijective");
od;

vm:=List(ConjugacyClassesMaximalSubgroups(Hv),Representative);;
vA5:=Filtered(vm,h->Size(h)=60);;
Check179(Length(vA5)=2,"two vertex A5 classes");;
Check179(Set(List(vA5,h->SortedList(List(Orbits(h,Adj[1]),Length))))
 =[[1,10],[5,6]],"two neighbour actions");;
H6:=First(vA5,h->SortedList(List(Orbits(h,Adj[1]),Length))=[5,6]);;
six:=Set(First(Orbits(H6,Adj[1]),x->Length(x)=6));;
VertexBlocks:=Set(Orbit(Hv,six,OnSets));;
Check179(Length(VertexBlocks)=11 and ForAll(Combinations(Adj[1],2),t->
 Number(VertexBlocks,b->IsSubset(b,t))=3),"neighbour 2-(11,6,3) design");;
for t in Combinations(Adj[1],3) do
 n:=Number(VertexBlocks,b->IsSubset(b,t));
 Check179(n in [1,2] and ((n=2)=ForAny(Pet,p->IsSubset(p,Union([1],t)))),
  "Proposition 3.4 triple criterion");
od;
TripleAction:=function(t,g) return [t[1]^g,OnSets(t[2],g),OnSets(t[3],g)]; end;;
Triples:=Set(Orbit(G,[1,Difference(Adj[1],six),six],TripleAction));;
Check179(Length(Triples)=2926,"vertex-block orbit");;
orbs:=[];;
for v0 in six do
 e:=Set([1,v0]); p:=Pet[Position(Edges,e)];
 h:=Intersection(H6,Stabilizer(G,e,OnTuples));
 orbs:=Orbits(h,p);
 Check179(Size(h)=10 and SortedList(List(orbs,Length))=[5,5],"Proposition 3.5 D10");
 Check179(Number(orbs,x->Set(x) in White)=1 and Number(orbs,x->Set(x) in Orange)=1,
  "Proposition 3.5 pentagon colours");
 Check179(Number(Quads,q->q[3]=e and q[1] in List(orbs,Set)
  and q[2] in List(orbs,Set))=1,"Proposition 3.5 unique quadruple");
od;
Print("AUXILIARIES Petersen completeness, corrected edge rule, vertex blocks and
↪  Propositions 3.4/3.5 PASS\n");

# The retained Module C seed finder; its complete original certificate remains
# separately bound and is not replaced by this copied interface.
FindSeed179:=function(K)
 local a,b,c,d,iv,bs;
 a:=GeneratorsOfGroup(K)[1];
 Check179(Order(a)=2,"ambient first generator");
 iv:=Filtered(Elements(Centralizer(K,a)),x->Order(x)=2);
 for d in iv do
  if d=a then continue;fi;
  bs:=Filtered(Elements(Centralizer(K,d)),x->Order(x)=2 and Order(a*x)=5);
  for c in iv do
   if Order(c*d)<>5 then continue;fi;
   for b in bs do
    if Order(b*c)=3 and (a*b*c)^5=One(K)
      and ((d*c*b)^5*a)^3=One(K) and Group(a,b,c,d)=K then
     return [a,b,c,d];
    fi;
   od;
  od;
 od;
 Error("no retained source seed");
end;;
s:=FindSeed179(G);; w:=(s[2]*s[3]*s[4])^5;;
Marks:=[s,[s[1],w*s[2],s[3],s[4]]];;
Parabolics:=List(Marks,t->List([1..4],i->Group(t{Difference([1..4],[i])})));;
FixQuad179:=function(h) return
↪  Filtered(Quads,q->ForAll(GeneratorsOfGroup(h),g->QuadAction(q,g)=q)); end;;
BaseObjects:=[];; hs:=[];;
for mode in [1..2] do
 hs:=Parabolics[mode];
 f0:=Filtered(Edges,e->ForAll(GeneratorsOfGroup(hs[1]),g->OnSets(e,g)=e));
 f1:=FixQuad179(hs[2]); f2:=FixQuad179(hs[3]);
 if mode=1 then
  f3:=Filtered(Triples,t->ForAll(GeneratorsOfGroup(hs[4]),g->TripleAction(t,g)=t));
 else
  f3:=Filtered(V,v->ForAll(GeneratorsOfGroup(hs[4]),g->v^g=v));
 fi;
 Print("BASE_CANDIDATES mode ",mode," counts ",List([f0,f1,f2,f3],Length),"\n");
```

```
 Check179(ForAll([f0,f1,f2,f3],x->Length(x)=1),"unique source object fixed by each
 ↪  parabolic");
 Add(BaseObjects,[f0[1],f1[1],f2[1],f3[1]]);
od;
Print("STAGE_OBJECTS_PASS\n");

Opposite179:=function(e,f,p)
 return Length(e)=2 and Length(f)=2 and IsSubset(p,e) and IsSubset(p,f)
  and e[2] in Adj[e[1]] and f[2] in Adj[f[1]]
  and Intersection(e,f)=[]
  and ForAll(e,u->ForAll(f,v->not v in Adj[u]));
end;;
# Literal source predicates, with source ranks 0..3 represented by 1..4.
Incident179:=function(mode,i,j,x,y)
 local p,inter,v;
 Check179(i<j,"ordered source types");
 if i=1 then
  p:=Pet[Position(Edges,x)];
  if j=2 then
   return IsSubset(p,y[1]) and x in y[4];
  elif j=3 then
   if mode=1 then inter:=Intersection(p,y[2]);
   else inter:=Intersection(p,y[1]); fi;
   return IsSubset(p,y[3]) and Opposite179(y[3],inter,p);
  elif mode=1 then
   return y[1] in p and Length(Intersection(y[3],p))=3;
  else
   return ForAll(x,u->Dist[u][y]=2)
    and ForAny(White,w->IsSubset(w,Union(x,[y])));
  fi;
 elif i=2 then
  if j=3 then
   if not IsSubset(x[1],y[3]) then return false; fi;
   if mode=1 then
    return Length(Intersection(x[1],y[2]))=1 and Length(Intersection(x[2],y[1]))=1;
   else
    return Length(Intersection(x[1],y[1]))=1 and Length(Intersection(x[2],y[2]))=1;
   fi;
  elif mode=1 then
   return y[1] in x[1] and Length(Intersection(y[3],x[1]))=2
    and Length(Intersection(y[2],x[2]))=1;
  else return y in x[1]; fi;
 elif mode=1 then
  return y[1] in x[3] and Length(Intersection(y[3],x[3]))=1;
 else return y in x[3]; fi;
end;;
RepairedIncident179:=function(mode,i,j,x,y)
 local wimage,oimage;
 if not Incident179(mode,i,j,x,y) then return false; fi;
 if mode=1 and i=3 and j=4 then
  wimage:=Set(List(x[1],v->Intersection(Adj[y[1]],Adj[v])[1]));
  oimage:=Set(List(x[2],v->Intersection(Adj[y[1]],Adj[v])[1]));
  return Set([wimage,oimage])=Set([y[2],Difference(y[3],x[3])]);
 fi;
 return true;
end;;
Actions179:=[OnSets,QuadAction,QuadAction,TripleAction];;
base:=[];; i:=0;; j:=0;; neighbours:=[];; extra:=[];;
for mode in [1..2] do
 hs:=Parabolics[mode]; base:=BaseObjects[mode];
 if mode=1 then Faces:=[Edges,Quads,Quads,Triples];
 else Faces:=[Edges,Quads,Quads,V]; Actions179[4]:=OnPoints; fi;
 Check179(List(hs,Size)=[[120,20,20,60],[120,20,20,660]][mode],"retained parabolic orders");
 for i in [1..4] do
  Check179(Stabilizer(G,base[i],Actions179[i])=hs[i],"exact source stabilizer");
  Check179(IsTransitive(G,Faces[i],Actions179[i]),"source face transitivity");
 od;
 for ij in Combinations([1..4],2) do
  i:=ij[1]; j:=ij[2];
  Check179(RepairedIncident179(mode,i,j,base[i],base[j]),"base pair incident");
  literal:=Filtered(Faces[j],y->Incident179(mode,i,j,base[i],y));
  neighbours:=Filtered(Faces[j],y->RepairedIncident179(mode,i,j,base[i],y));
  orbit:=Set(Orbit(hs[i],base[j],Actions179[j]));
  if mode=1 and i=3 and j=4 then
```

```
   Check179(Length(literal)=12 and Length(neighbours)=2,"printed Q rule versus repaired
   ↪  rule");
   Print("SOURCE_Q23_DEFECT literal neighbours ",Length(literal)," repaired
   ↪  ",Length(neighbours),
    " stabilizer_orbits
    ↪  ",SortedList(List(OrbitsDomain(hs[i],literal,Actions179[j]),Length)),"\n");
   Print("SOURCE_Q23_EXTRA_FLAG ",[base[i],First(literal,t->not t in neighbours)],"\n");
   extra:=First(literal,t->not t in neighbours);
   ext1:=Filtered(Quads,q->Incident179(1,2,3,q,base[3]) and Incident179(1,2,4,q,extra));
   Check179(Length(ext1)=0,"literal pair has no type-one extension");
   Print("SOURCE_Q23_EXTRA_FLAG_TYPE1_EXTENSIONS ",Length(ext1),"\n");
   Print("SOURCE_Q23_WITNESS_MEMBERSHIP ",[base[3] in Quads,extra in Triples,
    Incident179(1,3,4,base[3],extra)],"\n");
  else Check179(literal=neighbours,"other eleven relations unchanged"); fi;
  Check179(Set(neighbours)=orbit,"effective incidence equals complete stabilizer orbit");
  Check179(Length(orbit)=Index(hs[i],Intersection(hs[i],hs[j])),"orbit index");
  Print("INCIDENCE mode ",mode," types ",ij-[1,1]," neighbours ",Length(neighbours),
   " stabilizer ",Size(Intersection(hs[i],hs[j]))," PASS\n");
 od;
 Print("FACE_COUNTS mode ",mode," ",List(Faces,Length),"\n");
 Print("FLAG_STABILIZERS mode ",mode," ",List(Combinations([1..4]),
  js->Concatenation([js],List([Intersection(Concatenation([G],hs{js}))],
   h->[Size(h),StructureDescription(h)]))),"\n");
od;
Print("L068_P_AND_REPAIRED_Q_BRIDGE_PASS\n");
QUIT;
```